Laboration 2: Cellular Automata

Group Supermodels
Vootele Mets
Juan Rodriguez
Marco Malosti
Björk Lucas
Sofia Fernandes
David Weingut

May 4, 2026

UPPSALA
UNIVERSITET

Contents

1 Problem statement **2**

2 Theoretical Background **3**
- 2.1 Game of Life and Logic Gates . . . 3
  - 2.1.1 Game of Life as a discrete dynamical system . . . 3
  - 2.1.2 Logic gates in the Game of Life . . . 4
  - 2.1.3 Encoding of Inputs . . . 5
  - 2.1.4 Reading outputs . . . 5
  - 2.1.5 Initial conditions for the different gates . . . 5
- 2.2 Neuron Automaton Theory . . . 6
- 2.3 GHCA theory . . . 7

3 Model and parameters **8**

4 Numerical method **9**
- 4.1 Game of Life Numerics . . . 9
  - 4.1.1 Computational domain . . . 9
  - 4.1.2 Time evolution . . . 9
  - 4.1.3 Initial conditions . . . 9
  - 4.1.4 Logical gates . . . 10
  - 4.1.5 Simulation procedure and visualization . . . 10
  - 4.1.6 Debugging and validation . . . 10
- 4.2 Neuron Cellular Automaton . . . 10
- 4.3 GHCA Method . . . 11

5 Results **12**
- 5.1 Game of Life Results . . . 12
- 5.2 Neuron Automaton Results . . . 13
- 5.3 GHCA Results . . . 14

6 Interpretation **15**

7 Conclusion **16**

8 Use of LLM tools **16**
- 8.1 Neuron cellular automaton . . . 16

1. Problem statement

In this lab we will be investigating classical examples of cellular automata. Cellular automata are ...

The goal of the lab is to ...

All of these tasks will give us an overview of the characteristics in cellular automata ...

2. Theoretical Background

In this section we present the theory needed to solve the tasks of the lab.

2.1. Game of Life and Logic Gates

2.1.1. Game of Life as a discrete dynamical system

Conway's Game of Life can be viewed as a discrete dynamical system on a rectangular lattice $X^{(t)} \in \{0, 1\}^{m \times n}$, where each entry $X_{ij}^{(t)}$ represents the state of the cell at site (i, j) at time t. The evolution of the system is determined by update rules that specify the state of each cell at the next time step, $X_{ij}^{(t+1)}$.

Definition 2.1 – Conway's rules [2]

1. **Survival**: a live cell stays alive if it has 2 or 3 live neighbors.
2. **Birth**: a dead cell becomes alive if it has exactly 3 live neighbors.
3. **Death by isolation**: a live cell dies if it has fewer than 2 live neighbors.
4. **Death by overcrowding**: a live cell dies if it has more than 3 live neighbors.
5. **Neighborhood**: the neighbors of a cell are the 8 surrounding cells.

Let $N_{ij}^{(t)}$ denote the degree of node $X_{ij}^{(t)}$ at time t. Since each cell has eight neighbors, the relevant directions are the cardinal and intercardinal directions. Self-interaction is excluded, so the evolution of a cell depends only on its neighborhood. The quantity $N_{ij}^{(t)}$ can be computed as

$$N_{ij}^{(t)} = \sum_{p=-1}^{1} \sum_{q=-1}^{1} K_{p+2,q+2} X_{i+p,j+q}^{(t)}, \tag{2.1}$$

where K is a 3×3 kernel. Equivalently,

$$N^{(t)} = K * X^{(t)}, \tag{2.2}$$

where $K * X^{(t)}$ denotes the convolution of the lattice $X^{(t)}$ with the kernel K.

The update rule can then be written as

$$X_{ij}^{(t+1)} = \begin{cases} 1, & \text{if } X_{ij}^{(t)} = 1 \text{ and } N_{ij}^{(t)} \in \{2, 3\}, \\ 1, & \text{if } X_{ij}^{(t)} = 0 \text{ and } N_{ij}^{(t)} = 3, \\ 0, & \text{otherwise.} \end{cases} \tag{2.3}$$

In vectorized form, this becomes

$$X^{(t+1)} = \mathbb{1}\left(N^{(t)} = 3\right) + X^{(t)} \odot \mathbb{1}\left(N^{(t)} = 2\right), \tag{2.4}$$

and we use dirichlet boundary conditons or zero padding to handle the adjacency of boundary nodes.

2.1.2. Logic gates in the Game of Life

In classical computation, basic logic gates such as AND, OR, and NOT act on one or two input streams and produce a single output stream. For example,

$$\begin{array}{c} \texttt{01001010} \\ \underline{\texttt{10101010}} \\ \texttt{00001010} \\ \texttt{AND} \end{array} \qquad \begin{array}{c} \texttt{01001010} \\ \underline{\texttt{10101010}} \\ \texttt{11101010} \\ \texttt{OR} \end{array} \qquad \begin{array}{c} \underline{\texttt{01001010}} \\ \texttt{10110101} \\ \\ \texttt{NOT} \end{array}$$

Such gates implement a local transformation of binary information.

In the Game of Life, which is a discrete dynamical system evolving on a lattice, one may view logic gates from a different perspective: not as abstract Boolean operators, but as spatially embedded computational devices. The initial state is a geometric configuration, and the update rule propagates that configuration through time (changes its geometry from one to another). From this viewpoint, the construction of a logic gate becomes a problem of geometric constraint and synchronization. The central question is which initial lattice configuration (i.e., which geometry) $X^{(0)}$ can, under Conway's rules, evolve so as to realize deterministic, time-dependent binary information processing in the form of `AND`-, `OR`-, and `NOT`-gates?

First, we need to construct an initial geometry that evolves, under Conway's rules, into a space-time signal. For this we could use a train of *gliders* that travels periodically and diagonally across the lattice [3]. To get an arbitrary length of space-time signals we migh use *guns* that generate gliders [4]. The trajectories along which gliders evolve act as rigid wiring, thereby constraining the direction of information flow. In the absence of any interference, the signal propagates persistently along these paths. To obtain a richer computational structure, this simple stream of gliders (interpretable as a sequence of 1s) must be complemented with the ability to represent absence (0s) as meaningful information. Furthermore, one requires mechanisms for interaction between multiple input streams such that they can be combined to produce a third output stream, with the resulting configurations remaining composable within larger constructions.

It is possible to realize [1] composable logic gates in the Game of Life by spatially arranging glider guns, eaters, and collision configurations such that controlled streams of gliders encode binary information. In this setting, glider presence corresponds to logical `1`, while the absence of a glider stream corresponds to `0`. By appropriately translating and positioning glider guns, and by placing eaters as stabilizing and filtering structures, one can engineer deterministic collision patterns that implement logical operations. Moreover, these constructions are required to be composable, in the sense that the output of one gate can be routed as the input to another without loss of computational fidelity.

2.1.3. Encoding of Inputs

See `GoLogic.py` and the different input lanes, maybe put images of the gun, eater and glider i don't know.

2.1.4. Reading outputs

Reading outputs should be over a 4×4 bounding box, where a glider fits in? The glider in the output has an intercardinal direction. Synchronization?

2.1.5. Initial conditions for the different gates

See `GoLogic.py` for the different gates.

2.2. Neuron Automaton Theory

FOR THEORY OF GHCA...

2.3. GHCA theory

GHCA THEORY HERE

3. Model and parameters

MODEL AND PARAMETERS?

4. Numerical method

In this section we will describe the numerical procedures used in our code and state the relevant parameters to reproduce our findings.

4.1. Game of Life Numerics

4.1.1. Computational domain

The computational domain is represented as a two-dimensional rectangular lattice $X \in \{0, 1\}^{n \times n}$, where each entry corresponds to one cell of the automaton. The state variable is stored as an 8-bit unsigned integer array, `np.uint8`, in order to minimize memory overhead while preserving the binary structure of the model. In the simulations reported here, we use a square grid with $n = 400$.

4.1.2. Time evolution

The time evolution is implemented as a composition of a linear convolution operation and a point-wise non-linear update. The discrete convolution is computed using `scipy.ndimage.convolve`. A fixed 3×3 kernel

$$K = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix} \tag{4.1}$$

is used to compute the Moore neighbourhood sum

$$N^{(t)} = K * X^{(t)}. \tag{4.2}$$

The central entry of the kernel is zero, so the cell itself is excluded from the neighbour count.

The Game of Life update rule is then applied by vectorized logical operations:

$$X^{(t+1)} = \mathbb{1}\left(N^{(t)} = 3\right) \vee \left[X^{(t)} \wedge \mathbb{1}\left(N^{(t)} = 2\right)\right]. \tag{4.3}$$

Thus, a dead cell becomes alive if it has exactly three live neighbours, while a live cell survives if it has two or three live neighbours. The resulting Boolean array is cast back to `np.uint8`.

The update function allows both zero-padding boundaries, implemented with `mode="constant"`, and periodic boundaries, implemented with `mode="wrap"`. For the logical-gate simulations, zero-padding boundaries are used, since the gates are placed away from the domain boundary and the construction is intended to represent a finite portion of an otherwise empty Game of Life plane.

4.1.3. Initial conditions

Initial conditions are constructed programmatically in `GoLogic.py`. The basic components are the Gosper glider gun and the eater, both stored in Run Length Encoding (RLE). The RLE strings are parsed into binary arrays using a custom parser and inserted into the computational domain at fixed coordinate offsets. This makes the construction reproducible, since each component is always placed at the same location for a given input condition.

The code also implements geometric transformations of these base components, including rotations and left-right or up-down reflections. This allows the same gun and eater patterns to be reused in different orientations. The helper functions `add_pattern` and `transform_pattern` are used to insert and transform the components before simulation.

In addition to programmatic construction, the script includes a function `load_live_cells(filename, n)`, which can read initial configurations from external text files. These files contain one live cell per line in the format `(x,y)`. The file is converted into an $n \times n$ binary array by setting `grid[y,x] = 1`. This was used to support the required coordinate-list representation of the initial gate configurations.

4.1.4. Logical gates

The logical gates `AND`, `OR`, and `NOT` are realized through spatial arrangements of glider guns and eaters. Signals are represented by streams of gliders emitted by Gosper glider guns. Logical input values are encoded by modifying the initial configuration: an input value of 1 corresponds to allowing the relevant signal to propagate, while an input value of 0 corresponds to placing an eater so that the corresponding signal is suppressed.

For the `AND` and `OR` gates, all four input combinations were simulated:

$$(A, B) \in \{(0, 0), (0, 1), (1, 0), (1, 1)\}.$$

For the `NOT` gate, the two possible input values were simulated:

$$A \in \{0, 1\}.$$

The output was read visually from the designated output lane. The presence of a propagating glider stream in this lane was interpreted as logical output 1, while its absence was interpreted as logical output 0.

4.1.5. Simulation procedure and visualization

For each gate, the corresponding initial condition is created on a 400×400 lattice and then iterated for `nsteps = 1000` time steps. The automaton is advanced explicitly by repeated application of the update function. For visualization, `matplotlib.pyplot.imshow` is used with a binary colour map, so that live and dead cells are directly represented as black and white pixels.

To compare input cases systematically, the simulations for each gate were displayed in a single window. The `AND` and `OR` gates were shown using a 2×2 subplot layout, with one panel for each input pair. The `NOT` gate was shown using a 1×2 layout, with one panel for each input value. Each panel was labelled with the corresponding input condition, and the figure title displayed the current time step. The axes were restricted to the active region of the lattice so that the glider interactions and output signal were easier to observe.

4.1.6. Debugging and validation

Since the Game of Life update rule is not invertible, reverse time stepping is not defined. Validation therefore relied on forward simulations and direct inspection of the time evolution. The simulation was run for each input condition, and the observed outputs were compared with the expected Boolean truth tables of the corresponding gates. The time-step label in the visualization made it possible to select representative snapshots showing the initial configuration, signal propagation, gate interaction, and final output behaviour.

4.2. Neuron Cellular Automaton

The script for the implementation of the neuron cellular automaton is `neuron.py`. The code is based on `gameOfLifeInteractiveToric.py`, which was available on the Studium page of the course.

Now, we will present how the code works. At first, the number of rows and columns of the grid as well the speed of the simulation are defined. Additionally, the rules text is added. This text is displayed when clicking the ”rules”-button while running the script. Next, we have implemented the new function `count_neighbors_by_state` that can calculate the number of neighbours in a certain state. This function is necessary as we will be working with three different states instead of two. The function takes the grid and the state that we want to determine the number of neighbours for as arguments. The neighbours for each gridpoint are found using `numpy.roll` to include the periodic boundary conditions. As the next step we have updated the `lifeStep` function. Now, we first create the grids of neighbours in states 1 and 2 and apply the rules afterwards. State 0 represents the resting neurons, state 1 the ready neurons and state 2 the firing neurons. The function then returns the updated grid.

In the constructor of the `GameOfLifeApp` class the visualisation is defined. This includes the buttons that are displayed and their actions. The time step for the continuous run is defined as well.

The following parameters are fixed in the code: the number of rows and columns is set to 30 each. For the random population of the grid, the probability for state 0 is 0.7, for state 1 it is 0.2 and for state 2 it is 0.1. Initially, all cells are in state 0 (resting). Clicking once sets them to state 1 and clicking another time to state 2.

4.3. GHCA Method

FOR IMPLEMENTATION OF GHCA...

5. Results

5.1. Game of Life Results

FOR RESULTS OF Game of Life...

5.2. Neuron Automaton Results

FOR RESULTS OF NEURON AUTOMATON...

5.3. GHCA Results

FOR RESULTS OF GHCA...

6. Interpretation

INTERPRETATION GOES HERE.

7. Conclusion

CONCLUSION GOES HERE

8. Use of LLM tools

8.1. Neuron cellular automaton

The code in `neuron.py` is based on `gameOfLifeInteractiveToric.py` from the lecture. We used ChatGPT to modify the code. The prompt was ”I would like to change the rules of the game. I need to have three different states for each cell now and new rules, how can I implement this?” and additionally added the original code. This returned good instructions on where the code needs modification. Additionally, we looked up the documentation of the functions used by ChatGPT and made some modifications to its suggestions.

Files submitted

The following files have been submitted with this report:

1. `common.py`
2. `periodicPoints.py`
3. `phasePortrait.py`
4. `bifurcationCascade.py`

References

[1] Alex Bellos. Game of life: Logic gates.

[2] Jordi-Lluís Figueras. Interactive conway's game of life board for classroom demonstrations, 2026.

[3] LifeWiki. Glider.

[4] LifeWiki. Glider gun.